

Introduction to Reinforcement Learning

From Fundamentals to Flappy Bird

CPCS-331 Workshop Team

KAUST Academy AI Summer School Alumni

November 29, 2025

The Hook

What if machines could learn like us?

"No manual. No map. Just trial and error."

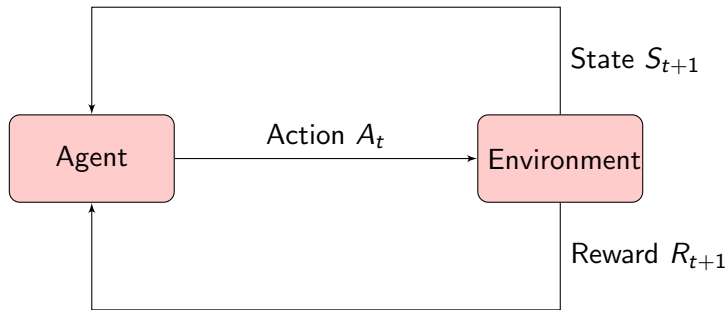


(Imagine a robot falling over and getting up again)

What is Reinforcement Learning?

- **Reinforcement Learning (RL)** is a computational approach to learning from interaction.
- **Goal:** Maximize a numerical reward signal over time.
- **Key Difference:** The agent is **not** told which action to take, but must discover which actions yield the most reward by trying them.

The Agent-Environment Loop



The Agent interacts with the Environment at discrete time steps t .

Key Terminology

- **Agent:** The learner and decision maker.
- **Environment:** Everything outside the agent.
- **Action (A):** What the agent does (e.g., Move Left).
- **State (S):** The current situation (e.g., Board position).
- **Reward (R):** Immediate feedback signal (Scalar).

The Reward Hypothesis

Sutton & Barto says:

"That all of what we mean by goals and purposes can be well thought of as the maximization of the expected value of the cumulative sum of a received scalar signal (called reward)."

Example: Chess

- Win: +1
- Lose: -1
- Draw/Intermediate moves: 0

Question Time!

Scenario: Dog Training

You are teaching your dog to sit. You say "Sit", and if it sits, you give it a treat.

Question Time!

Scenario: Dog Training

You are teaching your dog to sit. You say "Sit", and if it sits, you give it a treat.

- **Q:** Who is the Agent?

Question Time!

Scenario: Dog Training

You are teaching your dog to sit. You say "Sit", and if it sits, you give it a treat.

- Q: Who is the Agent? **The Dog.**
- Q: What is the Environment?

Question Time!

Scenario: Dog Training

You are teaching your dog to sit. You say "Sit", and if it sits, you give it a treat.

- Q: Who is the Agent? **The Dog.**
- Q: What is the Environment? **You / The Room.**
- Q: What is the Reward?

Question Time!

Scenario: Dog Training

You are teaching your dog to sit. You say "Sit", and if it sits, you give it a treat.

- Q: Who is the Agent? **The Dog.**
- Q: What is the Environment? **You / The Room.**
- Q: What is the Reward? **The Treat / "Good Boy".**
- Q: What is the Action?

Question Time!

Scenario: Dog Training

You are teaching your dog to sit. You say "Sit", and if it sits, you give it a treat.

- Q: Who is the Agent? **The Dog.**
- Q: What is the Environment? **You / The Room.**
- Q: What is the Reward? **The Treat / "Good Boy".**
- Q: What is the Action? **Sitting / Barking / Jumping.**

Where does RL fit?

The AI Landscape:

- **Supervised Learning:** Learning with a teacher (Labels provided).
 - Input: Image $\rightarrow$ Output: "Cat" (Correct!)
- **Unsupervised Learning:** Finding hidden structure (Clustering).
 - Input: Customer Data $\rightarrow$ Output: Segments.
- **Reinforcement Learning:** Learning by interaction.
 - Input: State $\rightarrow$ Action $\rightarrow$ Reward?

Supervised

- Instructional feedback.
- "This is a cat."
- Independent data points.

Reinforcement

- Evaluative feedback.
- "That was good/bad (+10/-10)."
- Time matters (Sequential data).
- Feedback is often delayed.

RL vs Search Algorithms (Our Course)

We learned about **A***, **BFS**, **DFS**. How is RL different?

- **Search (e.g., A*):**
 - Requires a **Model** (We know the map, the rules, the costs).
 - **Offline Planning:** The agent "thinks" before it acts to find the path.
- **Reinforcement Learning:**
 - Often **Model-Free** (We don't know the map initially).
 - **Online Learning:** The agent learns the map/strategy by walking around and bumping into walls.

Comparison Table

Feature	Search (A^*)	RL (Q-Learning)
Planning	Offline (Before acting)	Online (While acting)
Model of World	Required	Not Required (Usually)
Knowledge Source	Given Rules	Interaction/Experience
Output	A Path	A Policy (Strategy)

BONUS: Can RL Plan?

Yes! Model-Based RL.

- Algorithms like **AlphaZero** combine RL with Search (MCTS - Monte Carlo Tree Search).
- The agent learns a model of the world, then plans inside its head using that model.

Question Time!

Question

Is A* an RL algorithm?

Question Time!

Question

Is A* an RL algorithm?

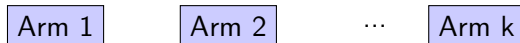
No.

- A* finds the optimal path given a graph. It doesn't "learn" a policy from scratch via rewards over time.

The k-Armed Bandit Problem

Imagine you are in a casino with k slot machines (bandits).

- Each machine pays out with a different (unknown) probability.
- You have N coins (turns).
- **Goal:** Maximize your total winnings.



Exploration vs Exploitation

This is the **core dilemma** of RL.

Exploitation

- Pull the arm you *think* is best.
- Maximize immediate reward.
- Risk: You might be stuck on a sub-optimal arm.

Exploration

- Pull a random arm / unknown arm.
- Gather information.
- Risk: You might lose money now to gain knowledge for later.

A Simple Solution: ϵ -Greedy

How do we balance this?

ϵ -Greedy Algorithm

Before every action, flip a biased coin:

- With probability $1 - \epsilon$: **Exploit** (Choose best known action).
- With probability ϵ : **Explore** (Choose random action).

Usually, ϵ is small (e.g., 0.1) and decays over time.

From Bandits to Context

Bandits have only **one state**.

- Life involves sequences.
- Your action now affects your position tomorrow.

Enter: Markov Decision Processes (MDPs).

The Markov Property

Definition

"The future depends only on the present, not the past."

$$P(S_{t+1}|S_t, A_t)$$

covers everything we need.

- Example: A Chess board position.
- You don't need to know the history of moves to decide the next best move. The physics of the game is fully contained in the current board configuration.

Components of an MDP

An MDP is defined by a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$:

- 1 **States ($\mathcal{S}$):** All possible situations.
- 2 **Actions ($\mathcal{A}$):** All possible things to do.
- 3 **Transition Probability ($\mathcal{P}$):** Physics of the world. If I jump, do I land or fall?
- 4 **Reward Function ($\mathcal{R}$):** The goal.
- 5 **Discount Factor (γ):** How much we care about the future.

The Discount Factor (γ)

Why do we discount future rewards? ($\gamma \in [0, 1]$)

- **Mathematical:** Keeps the sum of infinite rewards finite.
- **Uncertainty:** The future is uncertain. A treat now is better than a promised treat next year.
- $\gamma = 0$: Myopic Agent (Only cares about immediate reward).
- $\gamma \rightarrow 1$: Far-sighted Agent.

Policies and Value Functions

- **Policy (π):** The Agent's Brain.

$$\pi(a|s) = P(A_t = a | S_t = s)$$

Mapping from state to action. "If I see X, I do Y".

- **Value Function ($V(s)$):** "How good is it to be in this state?"

$$V_{\pi}(s) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \dots | S_t = s]$$

Tic-Tac-Toe

- State A: You have two Xs in a row. **High Value.**
- State B: Opponent has two Os in a row. **Low Value.**

The value function predicts the eventual outcome.

Action-Value Functions ($Q(s, a)$)

Often we want to know: "How good is it to take action A in state S ?"

- This is denoted as $Q(s, a)$.
- If we know the optimal $Q^*(s, a)$ for all actions, the optimal policy is easy:

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

- Just pick the action with the highest Q-value!

Tabular Q-Learning

Imagine a giant Excel sheet (Table).

- Rows = All possible States.
- Columns = All possible Actions.
- Cells = The Q-value (Expected future reward).

	Left	Right	Jump
State 1	0.5	2.0	-1.0
State 2	1.1	0.1	5.0
...	...	...	...

We update these numbers as we explore the world.

Question Time!

Problem

Why can't we use a table (Excel sheet) for a self-driving car?

Question Time!

Problem

Why can't we use a table (Excel sheet) for a self-driving car?

Too many states!

- The car sees a camera feed.
- Pixel values are continuous.
- Position is continuous.
- You cannot list every possible millisecond of driving as a row in a table.

The Curse of Dimensionality

- **Tic-Tac-Toe:** 5,000 states. (Table works).
- **Chess:** 10^{47} states. (Table impossible).
- **Go:** 10^{170} states. (More than atoms in universe).
- **Real World Images:** $256^{100 \times 100}$...

We need a better way.

Solution: Function Approximation

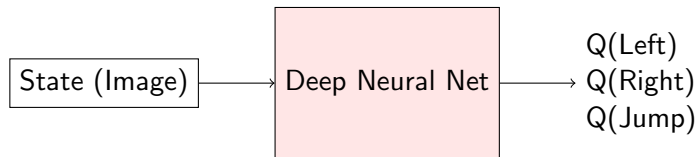
Instead of **memorizing** every state, we want to **generalize**.

- If State A looks like State B, the value should be similar.
- We approximate the Q-table with a function:

$$Q(s, a) \approx f(s, a, \theta)$$

Enter Neural Networks (Deep Q-Networks)

We use a **Neural Network** to approximate the Q-function.



The weights θ of the network replace the table entries.

Connecting to CNNs

You already know Convolutional Neural Networks (CNNs)!

- For Atari games, the input is raw pixels.
- We use CNN layers to extract features (edges, shapes, objects).
- These features are fed into dense layers to output Action Values.

Deep RL Challenges

Sounds easy? It's not.

- **Unstable:** RL with NNs is notoriously unstable.
- **Moving Target:** In Supervised Learning, the target (label) is fixed. In RL, our target (estimated Q) keeps changing as we learn.
- **Correlated Data:** Sequential video frames are highly correlated. NNs assume independent data (i.i.d).

BONUS: Experience Replay

How DQN solved the correlation problem:

Experience Replay Buffer

- 1 As the agent plays, save experiences (S, A, R, S') into a large memory buffer.
- 2 To train, sample a **random batch** from this memory.

This breaks the correlation between consecutive frames and stabilizes training.

Alternative: Policy Gradients

- DQN is **Value-Based** (learns Q-values).
- **Policy-Based** methods learn the Policy π directly.
- They adjust the probability of taking actions.
- If Action A led to good reward $\rightarrow$ Increase $P(A)$.
- Popular Algorithms: REINFORCE, **PPO** (Proximal Policy Optimization).

Family Tree of RL

- **Model-Free**

- Value-Based (DQN, Double DQN)
- Policy-Based (REINFORCE)
- Actor-Critic (A2C, PPO, SAC)

- **Model-Based**

- AlphaZero (MCTS + RL)
- World Models

History: Atari 2600

- **2013:** DeepMind introduced DQN.
- First agent to play generic Atari games at human level directly from pixels.
- Before this, RL relied on hand-crafted features tailored for each game.
- This sparked the "Deep RL" revolution.

The Environment Standard

To compare algorithms, we need standard benchmarks.

- **OpenAI Gym** (now maintained as **Gymnasium**).
- Provides a standard API for environments.
- Allows swapping environments with one line of code.

The Gym API

```
import gymnasium as gym

env = gym.make("CartPole-v1")
observation, info = env.reset()

for _ in range(1000):
    # Sample random action
    action = env.action_space.sample()

    # Step the environment
    obs, reward, terminated, truncated, info = env.step(action)

    if terminated or truncated:
        observation, info = env.reset()
```

Standard Benchmarks

- 1 **Classic Control:** CartPole, MountainCar (Simple physics).
- 2 **Atari:** Breakout, Pong, Space Invaders (Pixel input).
- 3 **MuJoCo:** Continuous control for robotics (Walking, hopping).

Action Spaces

Important distinction when choosing algorithms:

- **Discrete Space:** Finite set of actions.

Right, Jump

- Example: Super Mario, Atari.

- **Continuous Space:** Real-valued vectors.

Gas Pedal 0.8

- Example: Robotics, Self-driving cars.

Question Time!

Question

In Flappy Bird, is the action space Discrete or Continuous?

Question Time!

Question

In Flappy Bird, is the action space Discrete or Continuous?

Discrete.

- You either **Flap** or you **Don't Flap**.
- Binary action space $\{0, 1\}$.

Let's Build an Agent!

Objective: Teach an AI to play Flappy Bird.

- We will use the 'flappy-bird-gymnasium' library.
- We will use 'Stable Baselines3' (PPO algorithm).

State Representation

What does the agent see?

- We *could* use raw pixels (CNN), but training takes longer.
- For this workshop, we use **extracted coordinates**.

State Vector (12 dimensions):

- Horizontal distance to next pipe.
- Vertical distance to next pipe.
- Player vertical velocity.
- ...

Designing the Reward

We need to tell the agent what we want.

- **Alive:** +0.1 per frame. (Encourage survival).
- **Pass Pipe:** +1.0. (Encourage progress).
- **Death:** -100.0 or just Game Over. (Discourage crashing).

Choosing the Algorithm

We chose **PPO (Proximal Policy Optimization)**.

- It is the state-of-the-art for many tasks.
- Reliable and easy to tune compared to DQN.
- Available in Stable Baselines3.

The Code

```
import gymnasium as gym
import flappy_bird_gymnasium
from stable_baselines3 import PPO

# 1. Create Environment
env = gym.make("FlappyBird-v0")

# 2. Define Model (PPO)
model = PPO("MlpPolicy", env, verbose=1)

# 3. Train
model.learn(total_timesteps=100000)

# 4. Save
model.save("flappy_bird_expert")
```

DEMO TIME

1. Watch untrained agent (Random flailing).
2. Watch trained agent (God Mode).

RL is Everywhere

It's not just about video games anymore.

- Robotics
- Language Models (LLMs)
- Industrial Control
- Autonomous Driving

Sim-to-Real Transfer

- Training a robot in the real world is slow and dangerous (it falls and breaks).
- We train in a physics simulator (MuJoCo/Isaac Gym) using RL.
- We transfer the "brain" to the real robot.
- Example: Boston Dynamics, ANYbotics.

ChatGPT / Claude / Gemini

- Large Language Models are first trained to predict the next word (Supervised).
- Then they are fine-tuned using **RL from Human Feedback (RLHF)**.
- Humans rate the answers (Reward Model).
- PPO is used to optimize the LLM to generate high-reward answers (Helpful, Honest, Harmless).

Google Data Centers

- DeepMind used RL to control cooling fans and windows in Google data centers.
- Reduced cooling energy bill by **40%**.
- Action: Adjust fan speed. Reward: -Energy Usage.

Summary

- ① **RL** is learning by interaction (Action, State, Reward).
- ② **Exploration vs Exploitation** is the key trade-off.
- ③ **MDPs** provide the mathematical framework.
- ④ **Deep RL** (DQN, PPO) scales RL to complex worlds using Neural Networks.
- ⑤ **Applications** range from Atari to Robotics to ChatGPT.

Want to learn more?

- **Book:** *Reinforcement Learning: An Introduction* (Sutton & Barto).
- **Course:** Hugging Face Deep RL Course (Free, Online).
- **Library:** Stable Baselines3 (Python).

Thank You!

Questions?

(Bonus points for hard questions!)